

# INSTRUCTION.PDF

## Giáo án dự án môn học Edge AI and Introductory Physical AI

*From On-device Inference to Perception-Action Loops*

### Phiên bản chỉnh sửa

Bản này đã được chỉnh lại theo hướng: tăng độ rõ ràng của Physical AI, bổ sung mapping phần cứng - runtime - use case, chuẩn hóa benchmark end-to-end, thêm EdgeOps/MLOps cho thiết bị biên, và định vị Edge LLM như module nâng cao.

| Thông tin          | Nội dung                                                                                           |
|--------------------|----------------------------------------------------------------------------------------------------|
| Tên repo gợi ý     | edge-ai-physical-ai-systems                                                                        |
| Đối tượng          | Sinh viên/AI Engineer đã học Python, Machine Learning, Deep Learning và PyTorch cơ bản             |
| Định hướng         | Thực chiến triển khai AI trên thiết bị biên và mô phỏng vòng lặp Physical AI                       |
| Thời lượng đề xuất | 14 tuần chính khóa + 2 tuần mở rộng/bonus nếu cần                                                  |
| Sản phẩm cuối      | Repo hoàn chỉnh, demo inference, benchmark report, deployment notes, safety notes và final project |

## Mục lục nội dung

1. 1. Tóm tắt yêu cầu sau chỉnh sửa
2. 2. Mô tả tổng quan dự án
3. 3. Vị trí trong lộ trình AI
4. 4. Đối tượng và kiến thức đầu vào
5. 5. Mục tiêu đầu ra
6. 6. Triết lý thiết kế môn học
7. 7. Cấu trúc chương trình theo 5 phần
8. 8. Roadmap 20 chương
9. 9. Cấu trúc repo đề xuất
10. 10. Kế hoạch học 14 tuần
11. 11. Hardware - Runtime - Use case matrix
12. 12. Dataset và project anchor
13. 13. Chuẩn benchmark end-to-end
14. 14. EdgeOps, reliability và safety
15. 15. Final project và rubric
16. 16. Checklist triển khai repo

## 1. Tóm tắt yêu cầu sau chỉnh sửa

Bản giáo án gốc đã có nền tảng tốt cho Edge AI, nhưng cần chỉnh lại để rõ hơn về Physical AI và để phù hợp với một repo môn học có thể triển khai thực tế. Phiên bản này giữ tinh thần thực chiến, đồng thời tái cấu trúc thành một khóa học có 5 phần lớn: Edge AI Foundation, Model Deployment, Edge Applications, Physical AI, và Advanced Topics/Final Project.

- Tên khóa học được điều chỉnh thành “Edge AI and Introductory Physical AI” để phản ánh đúng trọng tâm: Edge AI là phần chính, Physical AI là phần nhập môn có định hướng mở rộng.
- Bổ sung nội dung Physical AI sâu hơn: state representation, decision layer, controller, actuator command, feedback loop, safety layer, simulation, sim-to-real và ROS2/robot simulator ở mức giới thiệu.
- Bổ sung bảng mapping giữa hardware, runtime, bài toán phù hợp và giới hạn triển khai.
- Bổ sung chuẩn benchmark end-to-end thay vì chỉ benchmark model-only latency.
- Bổ sung EdgeOps/MLOps tối thiểu cho thiết bị biên: model versioning, logging, monitoring, rollback và secure update.
- Định vị Edge LLM/VLM là phần nâng cao, không phải yêu cầu bắt buộc cho người mới học Edge AI.

## 2. Mô tả tổng quan dự án

Dự án nhằm xây dựng một repo môn học Edge AI / Physical AI hoàn chỉnh, tiếp nối sau các môn Machine Learning cho AI và Deep Learning cho AI. Nếu Machine Learning giúp người học hiểu cách mô hình học từ dữ liệu, Deep Learning giúp người học hiểu mạng neuron nhiều tầng và học biểu diễn, thì Edge AI / Physical AI đưa người học đến giai đoạn triển khai AI vào thế giới thật: thiết bị biên, camera, cảm biến, hệ thống nhúng, IoT, robot, dây chuyền sản xuất, thiết bị y tế, drone, xe tự hành nhỏ và các hệ thống tương tác vật lý.

Câu hỏi trung tâm của môn học:

Làm thế nào để đưa một mô hình AI từ notebook/server/GPU training environment ra chạy ổn định trên thiết bị thật, trong thời gian thực, với giới hạn về latency, memory, power, nhiệt độ, chi phí, quyền riêng tư và độ tin cậy?

Môn học không chỉ dạy model, mà bao phủ toàn bộ pipeline triển khai:

Repo môn học AI - Bản chỉnh sửa yêu cầu và giáo án

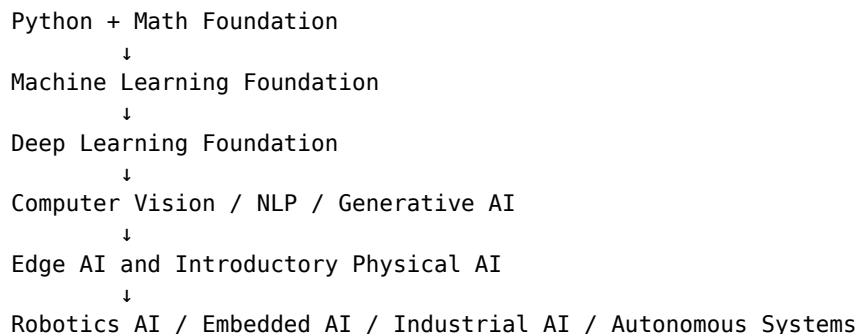
- Thu thập dữ liệu từ camera, microphone, sensor hoặc video stream.
- Huấn luyện hoặc fine-tune mô hình phù hợp với bài toán thực tế.
- Export model sang ONNX, TorchScript, TFLite hoặc runtime-specific format.
- Tối ưu bằng quantization, pruning, distillation, FP16/INT8 và runtime acceleration.
- Chạy inference bằng ONNX Runtime, TensorRT, OpenVINO, TFLite hoặc TensorFlow Lite Micro.
- Đo latency, FPS, throughput, memory, power, temperature và end-to-end response time.
- Kết nối model với camera, sensor, dashboard, API, MQTT hoặc actuator/simulator.
- Thiết kế vòng lặp perception -> state -> decision -> control -> action -> feedback.

### 3. Vì sao cần học Edge AI / Physical AI?

Trong nhiều năm, AI chủ yếu được huấn luyện và triển khai trên cloud/server mạnh. Tuy nhiên, nhiều ứng dụng thực tế không thể phụ thuộc hoàn toàn vào cloud vì cần phản hồi nhanh, dữ liệu liên tục, quyền riêng tư, khả năng hoạt động khi mất mạng, chi phí inference thấp và khả năng kiểm soát thiết bị vật lý.

- Camera/sensor tạo dữ liệu liên tục, gửi toàn bộ lên cloud gây tốn băng thông và tăng độ trễ.
- Robot, drone, xe tự hành nhỏ, thiết bị công nghiệp cần ra quyết định gần như tức thời.
- Dữ liệu hình ảnh, âm thanh, y tế hoặc sản xuất có thể nhạy cảm, cần xử lý cục bộ.
- Thiết bị biên thường bị giới hạn RAM, CPU/GPU/NPU, điện năng, nhiệt độ và chi phí phần cứng.
- Physical AI là bước tiếp theo: AI không chỉ nhận thức mà còn ra quyết định và hành động trong môi trường vật lý.

### 4. Vị trí trong lộ trình AI



Môn này nên học sau khi người học đã có nền Python, Machine Learning, Deep Learning và PyTorch cơ bản. Tuy nhiên, các chương đầu vẫn ôn lại inference, preprocessing, model.eval(), torch.no\_grad(), benchmark và export để người học chuyển từ tư duy training sang tư duy deployment.

### 5. Đối tượng người học và kiến thức đầu vào

| Nhóm người học           | Mục tiêu phù hợp                                                               |
|--------------------------|--------------------------------------------------------------------------------|
| AI Engineer              | Học cách đưa model ra production trên thiết bị thật, tối ưu latency và memory. |
| Sinh viên AI/CV/IoT      | Làm project camera AI, sensor AI, robot simulator hoặc TinyML.                 |
| Computer Vision Engineer | Deploy YOLO/classifier/segmentation lên edge device.                           |

| Nhóm người học             | Mục tiêu phù hợp                                                                  |
|----------------------------|-----------------------------------------------------------------------------------|
| Embedded/IoT Engineer      | Kết hợp AI inference với camera, sensor, MQTT, dashboard hoặc actuator.           |
| Người quan tâm Physical AI | Nắm nền tảng perception-action loop và simulation trước khi học robotics sâu hơn. |

Kiến thức đầu vào tối thiểu:

- Python, NumPy, Git và Linux command line cơ bản.
- PyTorch cơ bản: model, checkpoint, dataloader, train/eval mode.
- Hiểu classification, object detection hoặc segmentation ở mức nhập môn.
- Biết đọc ảnh/video bằng OpenCV là lợi thế.
- Không bắt buộc có sẵn Jetson/Raspberry Pi. Repo phải hỗ trợ mức 1 chạy trên laptop CPU/GPU trước.

## 6. Mục tiêu đầu ra của môn học

| Nhóm năng lực          | Người học cần đạt được                                                                                             |
|------------------------|--------------------------------------------------------------------------------------------------------------------|
| Tư duy hệ thống        | Phân biệt training, inference, deployment; phân tích latency, memory, power, bandwidth, cost và reliability.       |
| Inference deployment   | Load model, chạy inference PyTorch/ONNX Runtime, export ONNX/TorchScript/TFLite và validate output.                |
| Model optimization     | Hiểu và áp dụng FP16/INT8 quantization, PTQ, QAT nhập môn, pruning, distillation và trade-off accuracy-latency.    |
| Benchmarking           | Đo model-only latency, end-to-end latency, P50/P95/P99, FPS, memory, utilization và temperature nếu có.            |
| Edge applications      | Xây camera classifier, YOLO detection, sensor anomaly detection, dashboard/API/MQTT demo.                          |
| Physical AI foundation | Thiết kế vòng lặp sensors -> perception -> state -> decision -> controller -> action -> feedback với safety layer. |
| Deployment discipline  | Viết deployment notes, error analysis, rollback plan, monitoring/logging và safety notes.                          |

## 7. Triết lý thiết kế môn học

**Model chỉ là một phần của hệ thống:** Accuracy cao nhưng latency quá lớn, RAM quá cao hoặc thiết bị quá nóng thì không phù hợp triển khai.

**Đo đạc trước khi tối ưu:** Không tối ưu theo cảm tính. Luôn benchmark model size, latency, P95, FPS, RAM, utilization, power và temperature nếu đo được.

**Bắt đầu từ pipeline đơn giản:** Trước khi robot hoặc hệ thống phức tạp, người học phải làm được camera -> frame -> preprocessing -> inference -> postprocessing -> output.

**Accuracy - Latency - Power là tam giác đánh đổi:** Một project Edge AI tốt phải chỉ ra lựa chọn phù hợp nhất với ràng buộc thiết bị, không chỉ báo cáo accuracy.

**Physical AI cần vòng lặp hành động:** AI trên edge chưa chắc là Physical AI. Physical AI cần có sensor, state, decision, action, feedback và safety boundary.

**Safety và fallback là bắt buộc:** Khi AI tác động đến môi trường, cần có rule-based fallback, human override hoặc safety layer trước actuator.

## 8. Cấu trúc chương trình theo 5 phần

| Phần   | Tên phần                          | Vai trò                                                                                |
|--------|-----------------------------------|----------------------------------------------------------------------------------------|
| Part 1 | Edge AI Foundation                | Xây tư duy hệ thống, ràng buộc thiết bị, inference và benchmark.                       |
| Part 2 | Model Deployment                  | Export, runtime, acceleration và optimization.                                         |
| Part 3 | Edge Applications                 | Camera AI, object detection, sensor AI, TinyML và industrial inspection.               |
| Part 4 | Physical AI                       | Perception-state-decision-action, control loop, simulation, sim-to-real và ROS2 intro. |
| Part 5 | Advanced Topics and Final Project | Edge LLM/VLM, reliability, monitoring, safety và capstone project.                     |

## 9. Roadmap môn học theo từng chương

### Chương 0. Tổng quan Edge AI và Physical AI

**Mục tiêu:** Giúp người học hiểu Edge AI là gì, Physical AI là gì, khác gì cloud AI/server AI và vì sao đây là hướng quan trọng trong AI thực tế.

Nội dung chính:

- Cloud AI vs Edge AI.
- On-device inference.
- Real-time AI.
- Latency, throughput, FPS.
- Privacy và offline inference.
- Physical AI: perception, state, decision, action.
- Ứng dụng: camera AI, robot, drone, smart factory, health device, IoT.

Kết quả cần đạt:

- Giải thích được Edge AI và Physical AI.
- Phân biệt Edge AI với Embedded AI, TinyML, Robotics AI.
- Lấy được ví dụ thực tế cho từng nhóm ứng dụng.

Sản phẩm trong repo:

- docs/00\_intro\_edge\_physical\_ai.md
- figures/cloud\_vs\_edge.png

- figures/physical\_ai\_loop.png

## Chương 1. Thiết kế hệ thống Edge AI

**Mục tiêu:** Giúp người học phân tích bài toán Edge AI dưới góc nhìn hệ thống thay vì chỉ chọn model.

Nội dung chính:

- Input source: camera, microphone, sensor, video stream.
- Preprocessing, inference, postprocessing.
- Output: label, box, mask, command, alert.
- Latency budget và accuracy requirement.
- Hardware, power và deployment environment.
- Failure modes.

Kết quả cần đạt:

- Viết được system design ngắn cho một ứng dụng Edge AI.
- Xác định input, output, metric, latency target và rủi ro triển khai.

Sản phẩm trong repo:

- docs/01\_edge\_ai\_system\_design.md
- assignments/assignment\_01\_edge\_ai\_analysis.md

## Chương 2. Phần cứng cho Edge AI

**Mục tiêu:** Giúp người học hiểu các nhóm phần cứng phổ biến và cách chọn thiết bị phù hợp.

Nội dung chính:

- CPU, GPU, NPU/TPU accelerator.
- Microcontroller, Raspberry Pi, NVIDIA Jetson, Intel CPU/NPU, Google Coral TPU.
- Camera module và sensor.
- Cost, power, compute, ecosystem trade-off.

Kết quả cần đạt:

- Phân biệt microcontroller, single-board computer và edge GPU device.
- Biết khi nào dùng Raspberry Pi, Jetson, Intel OpenVINO hoặc microcontroller.

Sản phẩm trong repo:

- hardware\_notes/laptop\_cpu\_gpu.md
- hardware\_notes/raspberry\_pi.md
- hardware\_notes/nvidia\_jetson.md
- hardware\_notes/intel\_openvino.md

## Chương 3. Inference cơ bản: từ checkpoint đến dự đoán

**Mục tiêu:** Ôn lại khác biệt giữa training và inference, đồng thời xây inference script chuẩn.

Nội dung chính:

- Checkpoint loading.
- model.eval().
- torch.no\_grad().
- Preprocessing consistency.
- Single image và batch inference.
- Warm-up và latency cơ bản.

Kết quả cần đạt:

- Load một model PyTorch đã train.
- Chạy inference trên ảnh đơn/folder ảnh.
- Đo thời gian inference cơ bản.

Sản phẩm trong repo:

- notebooks/chapter\_01\_latency\_benchmarking.ipynb
- src/inference/infer\_pytorch.py

## Chương 4. Benchmarking: Latency, FPS, Memory, Throughput

**Mục tiêu:** Dạy cách đo hiệu năng nghiêm túc cho thiết bị biên.

Nội dung chính:

- Latency trung bình.
- P50/P95/P99 latency.
- Throughput và FPS.
- Warm-up iterations.
- Memory usage.
- CPU/GPU/NPU utilization.
- Model-only latency vs end-to-end latency.

Kết quả cần đạt:

- Viết benchmark script.
- Báo cáo latency đúng cách.
- Phân biệt inference time và total pipeline time.

Sản phẩm trong repo:

- src/benchmark/latency.py
- src/benchmark/memory.py
- src/benchmark/fps.py
- assignments/assignment\_03\_latency\_benchmark.md

## Chương 5. Export model: PyTorch -> ONNX -> Runtime

**Mục tiêu:** Giúp người học hiểu ONNX như định dạng trung gian để deploy qua nhiều runtime.

Nội dung chính:

- Vì sao cần export model.
- TorchScript.
- ONNX.
- Static shape vs dynamic shape.
- Opset.
- Validate exported model.
- Compare output PyTorch vs ONNX.
- Common export errors.

Kết quả cần đạt:

- Export PyTorch model sang ONNX.
- Kiểm tra ONNX model bằng ONNX Runtime.
- So sánh sai số output giữa PyTorch và ONNX.

Sản phẩm trong repo:

- notebooks/chapter\_02\_pytorch\_to\_onnx.ipynb
- labs/lab\_02\_export\_pytorch\_to\_onnx.ipynb
- src/export/export\_onnx.py

## Chương 6. ONNX Runtime cho Edge Inference

**Mục tiêu:** Dạy chạy inference bằng ONNX Runtime và hiểu execution provider.

Nội dung chính:

- CPUExecutionProvider.
- CUDAExecutionProvider.
- TensorRTEExecutionProvider ở mức khái niệm.
- OpenVINOExecutionProvider ở mức khái niệm.
- Session options.
- Input/output binding.
- Benchmark ONNX Runtime.

Kết quả cần đạt:

- Load ONNX model bằng ONNX Runtime.
- Chạy inference trên CPU/GPU.
- Đổi execution provider nếu môi trường hỗ trợ.

Sản phẩm trong repo:

- notebooks/chapter\_03\_onnxruntime\_inference.ipynb
- src/inference/infer\_onnxruntime.py

## Chương 7. TensorRT, OpenVINO và TFLite

**Mục tiêu:** Giới thiệu các runtime tối ưu thường dùng trong deployment.

Nội dung chính:

- TensorRT cho NVIDIA GPU/Jetson.
- OpenVINO cho Intel hardware.
- TFLite cho mobile/edge.
- TensorFlow Lite Micro cho microcontroller.
- FP16/INT8 inference.
- Calibration data.
- Khi nào chọn runtime nào.

Kết quả cần đạt:

- Hiểu vai trò inference engine.
- Biết workflow PyTorch -> ONNX -> TensorRT/OpenVINO.
- Biết workflow TensorFlow/Keras -> TFLite.

Sản phẩm trong repo:

- labs/lab\_04\_tensorrt\_or\_openvino.md
- src/inference/infer\_tensorrt.py
- src/inference/infer\_openvino.py

## Chương 8. Model Optimization: Quantization, Pruning, Distillation

**Mục tiêu:** Giúp người học hiểu các kỹ thuật giảm kích thước và tăng tốc model.

Nội dung chính:

- FP32, FP16, INT8.
- Dynamic/static quantization.
- Post-training quantization.
- Quantization-aware training nhập môn.
- Structured/unstructured pruning.
- Knowledge distillation.



- Accuracy-latency trade-off.

Kết quả cần đạt:

- Quantize một model đơn giản.
- So sánh model size, latency và accuracy trước/sau tối ưu.
- Hiểu khi nào quantization làm giảm accuracy.

Sản phẩm trong repo:

- notebooks/chapter\_04\_quantization\_ptq.ipynb
- src/optimization/quantization.py
- assignments/assignment\_04\_quantization.md

## Chương 9. Real-time Camera AI với OpenCV

**Mục tiêu:** Dạy pipeline camera inference thời gian thực.

Nội dung chính:

- OpenCV VideoCapture.
- Camera frame loop.
- Resize/normalize frame.
- Inference và postprocessing.
- Overlay kết quả.
- FPS counter.
- Frame skipping.
- Threading/multiprocessing cơ bản.

Kết quả cần đạt:

- Viết ứng dụng camera classification real-time.
- Hiển thị FPS.
- Tối ưu pipeline bằng resize, frame skip hoặc model nhỏ hơn.

Sản phẩm trong repo:

- notebooks/chapter\_06\_camera\_inference\_opencv.ipynb
- src/inference/camera\_loop.py
- labs/lab\_05\_realtime\_camera.md

## Chương 10. Object Detection trên thiết bị biên

**Mục tiêu:** Đưa người học từ classification sang object detection cho camera AI thực tế.

Nội dung chính:

- Bounding box.
- Confidence score.
- NMS.
- YOLO family ở mức thực hành.
- Lightweight detector.
- Input resolution vs FPS.
- Tracking cơ bản.
- Use case: people counting, product detection, safety helmet.

Kết quả cần đạt:

- Chạy YOLO trên ảnh/video.
- Export YOLO sang ONNX.
- Benchmark FPS.
- Tùy chỉnh confidence threshold.

Sản phẩm trong repo:

- projects/project\_02\_yolo\_realtime\_camera/
- notebooks/chapter\_07\_yolo\_edge\_detection.ipynb

## Chương 11. Edge AI cho sensor và time-series

**Mục tiêu:** Mở rộng Edge AI ngoài camera sang cảm biến công nghiệp, IoT và thiết bị đeo.

Nội dung chính:

- Sensor data.
- Time-series windowing.
- Feature extraction.
- Anomaly detection.
- Vibration/temperature/current/pressure sensor.
- Tiny model cho sensor.
- MQTT cơ bản.

Kết quả cần đạt:

- Chia dữ liệu sensor thành sliding windows.
- Train model phát hiện bất thường.
- Deploy inference loop giả lập stream sensor.
- Gửi alert qua MQTT/log dashboard.

Sản phẩm trong repo:

- projects/project\_03\_sensor\_anomaly\_detection/
- notebooks/chapter\_08\_sensor\_anomaly\_detection.ipynb

## Chương 12. TinyML và Microcontroller AI

**Mục tiêu:** Giới thiệu TinyML cho thiết bị cực nhỏ, nơi RAM, Flash và power rất hạn chế.

Nội dung chính:

- TinyML là gì.
- Microcontroller vs Raspberry Pi/Jetson.
- TensorFlow Lite Micro.
- Edge Impulse workflow ở mức khái niệm.
- Keyword spotting.
- Gesture recognition.
- Memory constraint.
- Quantization bắt buộc.

Kết quả cần đạt:

- Hiểu TinyML phù hợp bài toán nào.
- Biết workflow train -> quantize -> deploy microcontroller.
- Chạy demo keyword spotting/sensor classification nếu có board phù hợp.

Sản phẩm trong repo:

- projects/project\_05\_tinymml\_keyword\_spotting/
- notebooks/chapter\_09\_tinymml\_intro.ipynb

## Chương 13. Industrial Quality Inspection

**Mục tiêu:** Thiết kế một use case gần thị trường: kiểm tra lỗi sản phẩm bằng camera AI.

Nội dung chính:

- Visual inspection.
- Defect classification/detection/segmentation.

- Lighting và camera placement.
- False positive/false negative cost.
- Dataset collection.
- Edge deployment at factory line.
- Human-in-the-loop review.

Kết quả cần đạt:

- Thiết kế pipeline kiểm tra lỗi sản phẩm.
- Hiểu vấn đề dữ liệu công nghiệp: thiếu ảnh lỗi, imbalance, thay đổi ánh sáng.
- Chọn metric phù hợp.

Sản phẩm trong repo:

- projects/project\_04\_quality\_inspection\_ai/
- docs/13\_industrial\_quality\_inspection.md

## Chương 14. Physical AI: Perception -> State -> Decision -> Action

**Mục tiêu:** Giúp người học hiểu Physical AI như hệ thống AI có vòng lặp hành động trong môi trường vật lý.

Nội dung chính:

- Physical AI là gì.
- Robot perception.
- World state/state representation.
- Decision layer.
- Action layer.
- Control loop.
- Actuator.
- Closed-loop system.
- Rule-based decision vs learned policy.
- Safety layer.

Kết quả cần đạt:

- Mô tả pipeline sensors -> perception -> state -> decision -> controller -> actuator -> environment.
- Phân biệt perception-only AI và Physical AI có action/feedback.

Sản phẩm trong repo:

- docs/14\_physical\_ai\_loop.md
- src/physical\_ai/perception.py
- src/physical\_ai/decision.py
- src/physical\_ai/controller.py

## Chương 15. Control Loop, Safety Layer và Feedback

**Mục tiêu:** Bổ sung nền tảng điều khiển và an toàn ở mức nhập môn cho Physical AI.

Nội dung chính:

- Open-loop vs closed-loop.
- Controller cơ bản.
- PID ở mức khái niệm.
- Action command.
- Safety boundary.
- Human override.
- Fallback logic.

- Timeout và emergency stop trong simulator.

Kết quả cần đạt:

- Thiết kế controller đơn giản trong simulator.
- Thêm safety gate trước action command.
- Ghi log hành động và trạng thái lỗi.

Sản phẩm trong repo:

- labs/lab\_08\_robot\_simulation\_loop.md
- src/physical\_ai/controller.py
- src/physical\_ai/safety.py

## Chương 16. Simulation, Robotics và Sim-to-Real

**Mục tiêu:** Giới thiệu mô phỏng như bước quan trọng trước khi đưa Physical AI ra thế giới thật.

Nội dung chính:

- Vì sao cần simulation.
- Synthetic data.
- Domain randomization.
- Sim-to-real gap.
- Reinforcement learning ở mức định hướng.
- ROS/ROS2 giới thiệu.
- NVIDIA Isaac/Gazebo/PyBullet ở mức khái niệm.

Kết quả cần đạt:

- Hiểu vì sao không thể train/test robot hoàn toàn ngoài đời thật ngay từ đầu.
- Biết khái niệm sim-to-real.
- Xây toy simulation loop đơn giản bằng Python.

Sản phẩm trong repo:

- projects/project\_06\_physical\_ai\_simulation/
- notebooks/chapter\_10\_physical\_ai\_control\_loop.ipynb

## Chương 17. ROS2 / Robot Simulator Intro

**Mục tiêu:** Cung cấp cầu nối từ Physical AI nhập môn sang robotics engineering.

Nội dung chính:

- ROS2 concepts: node, topic, message, service ở mức giới thiệu.
- Publisher/subscriber.
- Camera/sensor topic.
- Command velocity/action topic.
- Simulator integration pattern.
- Không yêu cầu cài ROS2 bắt buộc trong khóa lỗi.

Kết quả cần đạt:

- Hiểu cách AI module giao tiếp với robot stack.
- Thiết kế mock ROS-like pub/sub bằng Python nếu chưa cài ROS2.

Sản phẩm trong repo:

- docs/17\_ros2\_robot\_sim\_intro.md
- labs/lab\_09\_mock\_ros\_loop.md

## Chương 18. Edge LLM và Multimodal AI trên thiết bị biên

**Mục tiêu:** Giới thiệu xu hướng mới, đặt ở module nâng cao để tránh quá tải.

Nội dung chính:

- On-device LLM.
- Small language models.
- Quantized LLM.
- Vision-language model ở edge.
- Prompt latency.
- Memory footprint.
- Local privacy.
- Robot instruction interface.
- Giới hạn thực tế khi chạy LLM trên edge.

Kết quả cần đạt:

- Hiểu khi nào chạy model nhỏ local, khi nào gọi cloud LLM.
- Thiết kế pipeline camera -> vision model -> local LLM decision ở mức khái niệm.

Sản phẩm trong repo:

- docs/18\_edge\_llm\_multimodal\_ai.md
- notebooks/chapter\_11\_edge\_llm\_intro.ipynb

## Chương 19. Security, Reliability, Monitoring và EdgeOps

**Mục tiêu:** Giúp người học hiểu vận hành hệ thống AI trên thiết bị biên sau khi demo chạy được.

Nội dung chính:

- Model failure.
- Sensor failure.
- Data drift.
- Privacy.
- Secure update.
- Logging.
- Monitoring latency/temperature/errors.
- Model versioning.
- Rollback.
- Fallback logic.

Kết quả cần đạt:

- Viết risk checklist cho Edge AI.
- Đề xuất fallback khi model không chắc chắn.
- Thiết kế log schema cho device inference.

Sản phẩm trong repo:

- docs/19\_security\_reliability\_edgeops.md
- deployment\_notes/runtime.md
- deployment\_notes/safety.md

## Chương 20. Final Project End-to-End

**Mục tiêu:** Người học hoàn thành một project triển khai AI trên thiết bị biên hoặc mô phỏng Physical AI hoàn chỉnh.

Nội dung chính:

- Problem statement.
- Device/simulator environment.
- Dataset.
- Model baseline.
- Export/optimization.
- Runtime.
- Benchmark.
- Metric.
- Demo script.
- Error analysis.
- Deployment note.
- Risk/safety note.

Kết quả cần đạt:

- Hoàn thành repo project tái lập được.
- Có benchmark và demo.
- Có báo cáo deployment/safety.

Sản phẩm trong repo:

- projects/final\_project\_template/
- reports/final\_project\_report\_template.md

## 10. Cấu trúc repo đề xuất

```
edge-ai-physical-ai-systems/
├── README.md
├── COURSE_OVERVIEW.md
├── SYLLABUS.md
├── ROADMAP.md
├── requirements.txt
├── environment.yml
├── docs/
│   ├── 00_intro_edge_physical_ai.md
│   ├── 01_edge_ai_system_design.md
│   ├── 02_hardware_for_edge_ai.md
│   ├── 03_model_inference_basics.md
│   ├── 04_benchmarking_and_profiling.md
│   ├── 05_pytorch_to_onnx.md
│   ├── 06_onnx_runtime.md
│   ├── 07_tensorrt_openvino_tflite.md
│   ├── 08_model_optimization.md
│   ├── 09_realtime_camera_ai.md
│   ├── 10_object_detection_edge.md
│   ├── 11_sensor_ai_timeseries.md
│   ├── 12_tinymml_microcontrollers.md
│   ├── 13_industrial_quality_inspection.md
│   ├── 14_physical_ai_loop.md
│   ├── 15_control_safety_feedback.md
│   ├── 16_simulation_sim_to_real.md
│   ├── 17_ros2_robot_sim_intro.md
│   ├── 18_edge_llm_multimodal_ai.md
│   ├── 19_security_reliability_edgeops.md
│   └── 20_final_project.md
├── notebooks/
├── labs/
├── assignments/
├── projects/
└── src/
```

```

├── data/
├── models/
├── export/
├── inference/
├── optimization/
├── benchmark/
├── edgeops/
├── physical_ai/
├── configs/
├── hardware_notes/
├── datasets/
├── experiments/
│   ├── logs/
│   ├── checkpoints/
│   ├── exported_models/
│   ├── benchmark_results/
│   └── reports/
├── figures/
└── reports/

```

## 11. Kế hoạch học 14 tuần

| Tuần | Nội dung                                        | Sản phẩm                                            |
|------|-------------------------------------------------|-----------------------------------------------------|
| 1    | Tổng quan Edge AI / Physical AI                 | Concept note + phân biệt Edge AI và Physical AI     |
| 2    | Edge system design, latency budget              | Assignment phân tích hệ thống                       |
| 3    | Hardware for Edge AI                            | Hardware comparison report + runtime selection note |
| 4    | Inference cơ bản và benchmark                   | PyTorch inference + benchmark script                |
| 5    | PyTorch -> ONNX                                 | Export model + validate output                      |
| 6    | ONNX Runtime                                    | ONNX inference demo + CPU/GPU benchmark             |
| 7    | TensorRT/OpenVINO/TFLite overview               | Runtime selection note + optional lab               |
| 8    | Quantization, pruning, distillation             | Optimization report                                 |
| 9    | Real-time camera AI                             | Camera classifier demo                              |
| 10   | Object detection edge                           | YOLO camera demo                                    |
| 11   | Sensor AI và TinyML                             | Sensor anomaly demo hoặc TinyML concept demo        |
| 12   | Physical AI loop                                | Perception-state-decision-action simulation loop    |
| 13   | Safety, reliability, EdgeOps, Edge LLM optional | Deployment design + risk checklist                  |
| 14   | Final project presentation                      | Report + demo + benchmark + deployment notes        |

## 12. Hardware - Runtime - Use case matrix

| Hardware                | Runtime chính                                   | Bài toán phù hợp                                               | Không phù hợp / lưu ý                                           |
|-------------------------|-------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------|
| Laptop CPU              | ONNX Runtime CPU, OpenVINO                      | Benchmark, demo nhỏ, classification, sensor AI                 | Không phù hợp real-time detection nặng nếu không có accelerator |
| Laptop GPU/NVIDIA GPU   | PyTorch, ONNX Runtime GPU, TensorRT             | CV model, YOLO, segmentation, prototyping trước khi lên Jetson | Cần chú ý khác biệt với edge device thật                        |
| NVIDIA Jetson           | TensorRT, ONNX Runtime GPU                      | Real-time camera AI, detection, industrial inspection          | Giới hạn RAM/power; cần tối ưu FP16/INT8                        |
| Raspberry Pi            | TFLite, ONNX Runtime nhẹ                        | Classifier nhẹ, sensor AI, IoT camera nhỏ                      | Không phù hợp YOLO lớn/VLM lớn                                  |
| Intel NUC/Intel CPU-NPU | OpenVINO                                        | Camera AI CPU/NPU, smart factory, retail, kiosk                | Cần kiểm tra operator/runtime support                           |
| Google Coral TPU        | TFLite Edge TPU                                 | Model TFLite INT8 tương thích Edge TPU                         | Bị giới hạn operator và quy trình compile                       |
| Microcontroller         | TensorFlow Lite Micro                           | Keyword spotting, sensor classification, gesture recognition   | Không phù hợp vision model lớn; RAM/Flash rất hạn chế           |
| Robot simulator         | Python loop, ROS2/Gazebo/PyBullet/Isaac concept | Physical AI loop, safety, feedback, action evaluation          | Không phản ánh hoàn toàn noise và latency đời thật              |

## 13. Dataset và project anchor

| Module                 | Dataset nên dùng                                          | Lý do chọn                                                    |
|------------------------|-----------------------------------------------------------|---------------------------------------------------------------|
| Camera classification  | CIFAR-10 hoặc custom webcam dataset                       | Nhẹ, dễ train, dễ export, phù hợp benchmark đầu khóa          |
| Object detection       | COCO subset hoặc safety helmet dataset                    | Gần ứng dụng camera AI, có bounding box và postprocessing NMS |
| Industrial inspection  | MVTec AD hoặc dataset defect đơn giản                     | Phù hợp kiểm tra lỗi, imbalance và anomaly/defect detection   |
| Sensor anomaly         | Synthetic stream, NASA bearing/vibration hoặc HAR dataset | Dạy windowing, time-series inference và alerting              |
| TinyML                 | Google Speech Commands hoặc gesture/HAR dataset           | Phù hợp keyword spotting, quantization và memory constraint   |
| Physical AI simulation | Toy grid-world hoặc object-reaching simulation            | Dễ mô phỏng state, decision, action, success rate và          |



| Module | Dataset nên dùng | Lý do chọn        |
|--------|------------------|-------------------|
|        |                  | collision/failure |

## 14. Chuẩn benchmark end-to-end

Benchmark trong khóa học phải được chuẩn hóa. Không được chỉ báo cáo “model chạy X ms” nếu pipeline thực tế còn capture frame, resize, normalize, postprocess, draw overlay, gửi alert hoặc publish command.

```
End-to-end latency = camera_capture
                    + preprocessing
                    + model_inference
                    + postprocessing
                    + visualization_or_output
                    + communication_or_action_delay
```

| Metric                  | Ý nghĩa                                   | Bắt buộc?                        |
|-------------------------|-------------------------------------------|----------------------------------|
| model_size_mb           | Dung lượng checkpoint/exported model      | Có                               |
| latency_mean_ms         | Độ trễ trung bình của model hoặc pipeline | Có                               |
| latency_p50_ms          | Median latency                            | Có                               |
| latency_p95_ms          | Độ trễ ở mức xấu nhưng thường gặp         | Có                               |
| latency_p99_ms          | Độ trễ tail latency                       | Khuyến nghị                      |
| fps_end_to_end          | FPS thực tế của toàn pipeline             | Có với camera                    |
| memory_peak_mb          | RAM/VRAM peak                             | Có                               |
| cpu_gpu_npu_utilization | Mức sử dụng tài nguyên                    | Khuyến nghị                      |
| temperature_celsius     | Nhiệt độ thiết bị                         | Khuyến nghị với edge device thật |
| power_watt              | Điện năng tiêu thụ                        | Tùy thiết bị                     |

Template benchmark tối thiểu:

```
import time
import numpy as np

def benchmark_inference(model_fn, inputs, warmup=20, repeat=100):
    for _ in range(warmup):
        _ = model_fn(inputs)

    times = []
    for _ in range(repeat):
        start = time.perf_counter()
        _ = model_fn(inputs)
        end = time.perf_counter()
        times.append((end - start) * 1000)

    times = np.array(times)
    return {
        "latency_mean_ms": float(times.mean()),
        "latency_p50_ms": float(np.percentile(times, 50)),
        "latency_p95_ms": float(np.percentile(times, 95)),
        "latency_p99_ms": float(np.percentile(times, 99)),
    }
```

```

    "fps_estimate": float(1000.0 / times.mean()),
}

```

## 15. EdgeOps, reliability và safety

Một hệ thống Edge AI thật không dừng lại ở demo chạy được. Repo môn học cần dạy người học tối thiểu cách vận hành model trên thiết bị biên: versioning, logging, monitoring, fallback và rollback.

- Mỗi model export cần có model\_version, runtime\_version, input\_shape, preprocessing\_config và checksum.
- Mỗi inference loop nên ghi log: timestamp, device\_id, model\_version, latency, confidence, output, error\_code.
- Camera/sensor failure cần được phát hiện bằng timeout, empty frame, invalid sensor range hoặc data drift heuristic.
- Nếu confidence thấp hoặc latency vượt ngưỡng, hệ thống cần fallback: cảnh báo người dùng, chuyển sang rule-based mode hoặc dừng action.
- Với Physical AI, action command phải đi qua safety gate trước khi gửi đến actuator hoặc simulator.
- Update model cần có rollback plan để quay lại model ổn định trước đó khi version mới lỗi.

```

device_log_schema:
  timestamp: str
  device_id: str
  model_version: str
  runtime: str
  input_source: camera_or_sensor
  latency_ms: float
  confidence: float
  prediction: str
  action_command: str_or_none
  temperature_celsius: float_or_none
  error_code: str_or_none
  fallback_triggered: bool

```

## 16. Final Project: yêu cầu và rubric

Final project cần chứng minh người học hiểu triển khai end-to-end, không chỉ train model. Project có thể là Edge AI thuần túy hoặc Physical AI simulation.

Nhóm đề tài gợi ý:

- Real-time camera image classifier.
- YOLO real-time object detection.
- Product defect detection demo.
- Sensor anomaly detection edge pipeline.
- TinyML keyword spotting hoặc gesture recognition.
- Edge dashboard cho camera AI.
- Physical AI simulator: perception -> state -> decision -> action.
- Edge LLM assistant cho camera monitoring ở mức bonus.

Pipeline bắt buộc:

17. Mô tả bài toán và bối cảnh triển khai.
18. Mô tả thiết bị hoặc môi trường simulator.
19. Dataset và preprocessing.
20. Model baseline.
21. Export/optimization hoặc runtime deployment.

- 22. Inference runtime.
- 23. Benchmark latency/FPS/memory, ưu tiên end-to-end latency.
- 24. Evaluation metric phù hợp.
- 25. Demo script.
- 26. Error analysis.
- 27. Deployment notes.
- 28. Risk/safety notes.

| Tiêu chí                                 | Điểm |
|------------------------------------------|------|
| Mô tả bài toán và ràng buộc edge rõ ràng | 10   |
| Dataset và preprocessing đúng            | 10   |
| Model baseline hợp lý                    | 10   |
| Export/optimization/inference runtime    | 15   |
| Benchmark latency/FPS/memory/end-to-end  | 15   |
| Metric đánh giá phù hợp                  | 10   |
| Demo chạy được                           | 10   |
| Error analysis                           | 10   |
| Deployment notes và safety notes         | 5    |
| Repo sạch, dễ tái lập                    | 5    |
| Tổng                                     | 100  |

## 17. Checklist triển khai repo môn học

- README.md giải thích rõ mục tiêu khóa học, yêu cầu đầu vào, cách cài đặt và cách chạy demo đầu tiên.
- Mỗi chương có docs, notebook/lab, assignment hoặc mini-project rõ ràng.
- Mỗi project có README riêng, config.yaml, script train/export/infer/benchmark và thư mục results mẫu.
- Có ít nhất một pipeline PyTorch -> ONNX -> ONNX Runtime hoàn chỉnh.
- Có ít nhất một pipeline real-time camera inference bằng OpenCV.
- Có ít nhất một bài benchmark trước/sau optimization.
- Có ít nhất một project sensor/time-series để không giới hạn khóa học trong Computer Vision.
- Có ít nhất một lab Physical AI simulation với perception-state-decision-action và safety gate.
- Có final\_project\_template và report\_template để người học làm capstone.
- Có deployment\_notes/hardware.md, runtime.md và safety.md cho từng project chính.

## 18. Kết luận

Phiên bản giáo án chỉnh sửa này phù hợp để dùng làm Instruction.pdf cho repo môn học Edge AI / Physical AI. Khóa học được định vị đúng hơn là “Edge AI and Introductory Physical AI”: phần Edge AI đủ sâu cho triển khai thực chiến, phần Physical AI đủ để người học hiểu vòng lặp hành động và chuẩn bị học tiếp robotics/embedded/autonomous systems. Khi triển khai repo thật, nên ưu tiên làm trước các lab cốt lõi: PyTorch inference, ONNX export, ONNX Runtime, benchmark, camera loop, quantization, sensor anomaly và physical simulation loop.